

Лабораторна робота №8

з теми «Огляд сучасного популярного
автоматизованого інструментарію»

Виконала:

ст. гр. ІП-22-1

Чемна (Микитин) О. І.

Перевірив:

Храбатин Р. І.

Мета: Дослідити та порівняти існуючі сучасні інструменти для створення автоматизованих рішень на основі JavaScript.

Короткі теоретичні відомості:

JavaScript часто використовується для створення автоматизованих тестів, оскільки має велику кількість інструментів для UI, API та E2E тестування. Для перевірки графічного інтерфейсу можна використовувати як повноцінні фреймворки, так і бібліотеки, які дають доступ до керування браузером.

Вибір інструменту залежить від складності проєкту, потреби у звітності, зручності підтримки тестів і рівня абстракції.

Puppeteer — це бібліотека Node.js, яка надає високорівневий API для керування браузером Chrome або Firefox через DevTools Protocol або WebDriver BiDi. Вона дозволяє відкривати сторінки, взаємодіяти з елементами, отримувати текст, перевіряти URL, робити скріншоти та виконувати браузерну автоматизацію. Puppeteer за замовчуванням може працювати у headless-режимі, тобто без відкритого графічного вікна браузера.

Gauge — це інструмент для автоматизованого тестування, який дозволяє описувати сценарії у більш читабельному форматі специфікацій. Він зручний тоді, коли потрібно відокремити опис тестового сценарію від технічної реалізації кроків. Для роботи з Gauge можна встановити CLI через npm командою `npm install -g @getgauge/cli`, а самі кроки в JavaScript реалізуються через `gauge.step`.

Завдання:

Мій варіант — 30.

Завдання перше:

Згідно варіанту провести невеличке дослідження можливостей заданих інструментів. Дослідження повинно включати наступні пункти: сильні сторони, слабкі сторони, типові задачі які інструмент вирішує без проблем (основне призначення). Порівняння оформити у вигляді таблиці:

Інструмент	Сильна сторона	Слабка сторона	Типові задачі
------------	----------------	----------------	---------------

Завдання друге:

Створити три UI тести використовуючи кожен з інструментів. За основу тестування можете брати попередні ресурси з якими ви вже добре знайомі. Проведіть аналіз вашого досвіду роботи з цими інструментами.

Завдання третє:

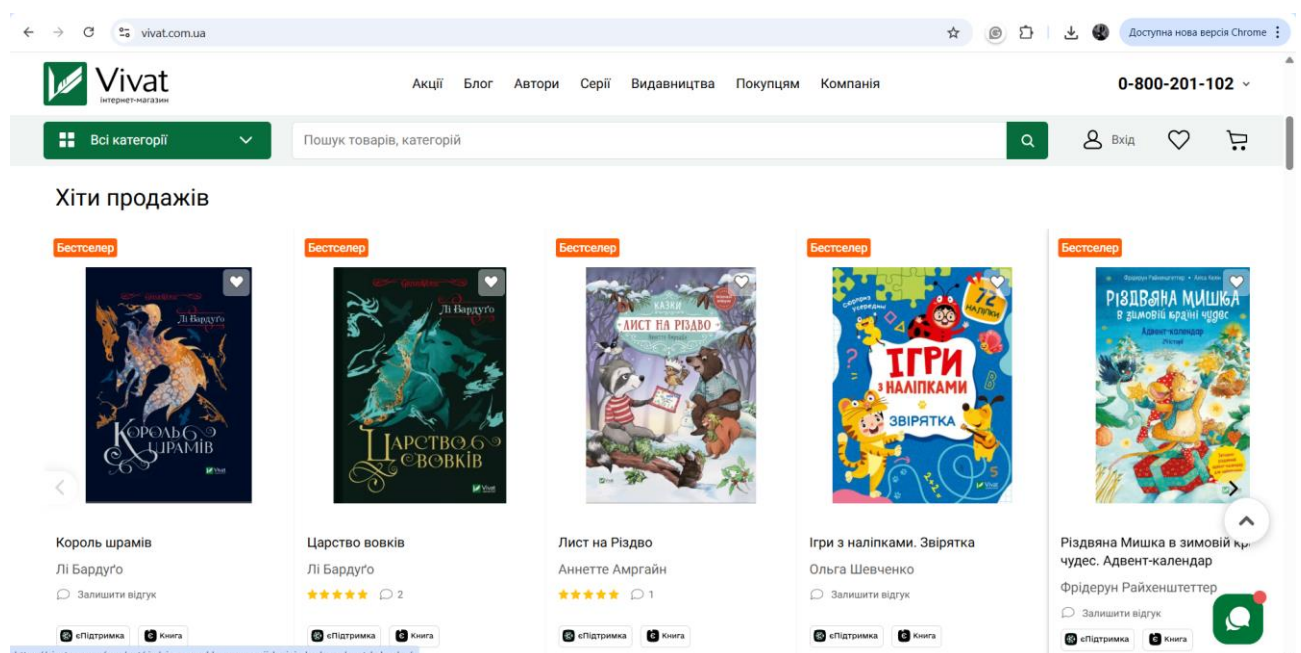
Завантажити код на власний/навчальний репозиторій GitHub, або інший схожий ресурс та поділитися посиланням.

На основі варіанту:

Puppeteer & Gauge

Виконання:

Для виконання лабораторної роботи я обрала сайт Видавництво Віват:



Я обрала цей сайт, оскільки він уже використовувався у попередніх лабораторних роботах і має зручні елементи для UI-тестування: головну сторінку, каталог книжок, посилання, текстовий контент і сторінки категорій.

Спочатку я провела коротке порівняння інструментів Puppeteer та Gauge:

Інструмент	Сильна сторона	Слабка сторона	Типові задачі
Puppeteer	Дає прямий контроль над браузером, дозволяє відкривати сторінки,	Має нижчий рівень абстракції, тому	UI-тести, E2E-тести, браузерна автоматизація,

	натискати елементи, отримувати текст і перевіряти інтерфейс	багато логіки потрібно писати вручну	перевірка сторінок, скріншоти
Gauge	Дозволяє описувати сценарії у вигляді зрозумілих специфікацій і відокремлювати опис тесту від коду	Потребує додаткової структури проєкту та окремої реалізації step definitions	BDD-подібні сценарії, assertance-тести, читабельні специфікації для команди

Після цього я створила окремий каталог для лабораторної роботи:

```
oksanka@debian:~/Documents/automated_testing$ mkdir lab-8-tools-comparison
oksanka@debian:~/Documents/automated_testing$ cd lab-8-tools-comparison
```

Далі я ініціалізувала Node.js-проєкт:

```
oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$ npm init -y
Wrote to /home/oksanka/Documents/automated_testing/lab-8-tools-comparison/package.json:

{
  "name": "lab-8-tools-comparison",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$
```

Після цього я встановила Jest і Puppeteer для першої частини роботи:

```
oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$ npm install --save-dev jest puppeteer
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it.
Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which
is much more comprehensive and powerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publiciz
ed security vulnerabilities, which have been fixed in the current version. Please update. Support f
or old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publi
cized security vulnerabilities, which have been fixed in the current version. Please update. Support
for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 358 packages, and audited 359 packages in 19s

48 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$
```

Далі я встановила Gauge CLI:

```
oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$ sudo npm install -g @getgauge/cli
[sudo] пароль до oksanka:

added 2 packages in 9s
npm notice
npm notice New major version of npm available! 10.9.7 -> 11.13.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.13.0
npm notice To update run: npm install -g npm@11.13.0
npm notice
oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$
```

Після цього я налаштувала файл package.json, щоб можна було запускати тести окремо для кожного інструменту:

```
> Debug
6   "scripts": {
7     "test:puppeteer": "jest puppeteer-tests",
8     "test:gauge": "gauge run specs"
9   },
10  "keywords": [
```

Реалізація UI-тестів за допомогою Puppeteer

Спочатку я створила файл для тестів Puppeteer. У файл puppeteer-tests/vivat.puppeteer.test.js я додала три UI-тести для сайту Видавництва Віват:

```
package.json U vivat.puppeteer.test.js U X
puppeteer-tests > vivat.puppeteer.test.js > ...
1  const puppeteer = require("puppeteer");
2
3  jest.setTimeout(90000);
4
5  const BASE_URL = "https://vivat.com.ua/";
6  const BOOKS_URL = "https://vivat.com.ua/category/knyhy/";
7
8  describe("Vivat UI tests with Puppeteer", () => {
9    let browser;
10   let page;
11
12   beforeEach(async () => {
13     browser = await puppeteer.launch({
14       headless: true,
15       args: ["--no-sandbox", "--disable-setuid-sandbox"],
16     });
17
18     page = await browser.newPage();
19
20     await page.setViewport({
21       width: 1366,
22       height: 768,
23     });
24   });
25
```

```

26   afterEach(async () => {
27     await browser.close();
28   });
29
30   test("should open Vivat home page", async () => {
31     await page.goto(BASE_URL, {
32       |   waitUntil: "networkidle2",
33     });
34
35     const currentUrl = page.url();
36
37     expect(currentUrl).toContain("vivat.com.ua");
38   });
39
40   test("should display title with Vivat text", async () => {
41     await page.goto(BASE_URL, {
42       |   waitUntil: "networkidle2",
43     });
44
45     const title = await page.title();
46
47     expect(title.toLowerCase()).toContain("vivat");
48   });

```

```

49
50   test("should open books category and display book-related content", async () => {
51     await page.goto(BOOKS_URL, {
52       |   waitUntil: "networkidle2",
53     });
54
55     const bodyText = await page.$eval("body", (body) => body.innerText);
56
57     expect(page.url()).toContain("/category/knyhy");
58     expect(bodyText.toLowerCase()).toContain("книг");
59   });
60 });
61

```

Перший тест перевіряв відкриття головної сторінки сайту Віват. Другий тест перевіряв, що заголовок сторінки містить інформацію про Vivat. Третій тест перевіряв відкриття категорії книг і наявність тексту, пов'язаного з книжками.

Після написання тестів я запустила Puppeteer-тести командою:

```

• oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$ npm run test:puppeteer

> lab-8-tools-comparison@1.0.0 test:puppeteer
> jest puppeteer-tests

PASS puppeteer-tests/vivat.puppeteer.test.js (17.822 s)
  Vivat UI tests with Puppeteer
    ✓ should open Vivat home page (7224 ms)
    ✓ should display title with Vivat text (5477 ms)
    ✓ should open books category and display book-related content (4838 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        17.859 s
Ran all test suites matching puppeteer-tests.
• oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$

```

Реалізація UI-тестів за допомогою Gauge

Після реалізації тестів на Puppeteer я перейшла до створення тестів за допомогою Gauge. Для цього я створила файл специфікації. У файл specs/vivat.spec я додала опис трьох UI-сценаріїв у форматі Gauge:

```
package.json U vivat.puppeteer.test.js U vivat.spec U X
specs > vivat.spec
1 # Vivat UI tests with Gauge
2
3 ## Home page should open successfully
4 * Open Vivat home page
5 * Page URL should contain "vivat.com.ua"
6
7 ## Home page title should contain Vivat
8 * Open Vivat home page
9 * Page title should contain "vivat"
10
11 ## Books category should contain book-related content
12 * Open Vivat books category page
13 * Page URL should contain "/category/knyhy"
14 * Page body should contain "книг"
```

Після цього я створила файл із реалізацією кроків. У файл tests/step_implementation.js я додала код, який реалізує кроки зі специфікації Gauge:

```
tests > step_implementation.js > ...
1 const assert = require("assert");
2 const puppeteer = require("puppeteer");
3
4 const BASE_URL = "https://vivat.com.ua/";
5 const BOOKS_URL = "https://vivat.com.ua/category/knyhy/";
6
7 let browser;
8 let page;
9
10 beforeScenario(async function () {
11   browser = await puppeteer.launch({
12     headless: true,
13     args: ["--no-sandbox", "--disable-setuid-sandbox"],
14   });
15
16   page = await browser.newPage();
17
18   await page.setViewport({
19     width: 1366,
20     height: 768,
21   });
22 });
23
24 afterScenario(async function () {
25   if (browser) {
26     await browser.close();
27   }
28 });
29
```

```

30 step("Open Vivat home page", async function () {
31     await page.goto(BASE_URL, {
32         waitUntil: "networkidle2",
33     });
34 });
35
36 step("Open Vivat books category page", async function () {
37     await page.goto(BOOKS_URL, {
38         waitUntil: "networkidle2",
39     });
40 });
41
42 step("Page URL should contain <arg0>", async function (arg0) {
43     const currentUrl = page.url();
44
45     assert.ok(
46         currentUrl.includes(arg0),
47         `Expected URL to contain ${arg0}, but got ${currentUrl}`,
48     );
49 });
50
51 step("Page title should contain <arg0>", async function (arg0) {
52     const title = await page.title();
53
54     assert.ok(
55         title.toLowerCase().includes(arg0.toLowerCase()),
56         `Expected title to contain ${arg0}, but got ${title}`,
57     );
58 });
59
60 step("Page body should contain <arg0>", async function (arg0) {
61     const bodyText = await page.$eval("body", (body) => body.innerText);
62
63     assert.ok(
64         bodyText.toLowerCase().includes(arg0.toLowerCase()),
65         `Expected page body to contain ${arg0}`,
66     );
67 });

```

У Gauge я описала сценарії більш читабельно, а технічну реалізацію винесла в окремий файл `step_implementation.js`. Завдяки цьому сценарії стали схожими на документацію, яку може зрозуміти не тільки розробник або тестувальник, а й інші учасники команди.

Перший сценарій Gauge перевіряв відкриття головної сторінки. Другий сценарій Gauge перевіряв заголовок сторінки. Третій сценарій Gauge перевіряв сторінку категорії книг.

Після написання специфікацій і кроків я запустила Gauge-тести командою:


```
• oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$ npm run test:gauche
> test:gauche
> gauche run specs

# Vivat UI tests with Gauge
## Home page should open successfully ✓✓
## Home page title should contain Vivat ✓✓✓
## Books category should contain book-related content ✓✓✓

Successfully generated html-report to => /home/oksanka/Documents/automated_testing/lab-8-tools-comparison/reports/html-report/index.html

Specifications: 1 executed    1 passed    0 failed    0 skipped
Scenarios:      3 executed    3 passed    0 failed    0 skipped

Total time taken: 19.367s
• oksanka@debian:~/Documents/automated_testing/lab-8-tools-comparison$
```

Аналіз досвіду роботи з інструментами

Після виконання тестів я порівняла власний досвід роботи з Puppeteer та Gauge.

Критерій	Puppeteer	Gauge
Зручність старту	Швидко встановлюється і одразу дозволяє писати браузерні тести	Потребує додаткового налаштування структури специфікацій і кроків
Читабельність тестів	Код зрозумілий для технічного спеціаліста	Сценарії дуже зручні для читання, бо описані майже природною мовою
Гнучкість	Дуже гнучкий, можна напяму керувати браузером	Гнучкість залежить від того, як добре описані step definitions
Підтримка	Усі дії знаходяться в коді, тому простіше швидко редагувати маленький проєкт	Для великих сценаріїв підтримка може бути зручнішою, бо сценарії відділені від реалізації
Найкраще використання	Технічні UI та E2E перевірки	Асептанс-тести, BDD-подібні сценарії, читабельна документація тестів

На практиці Puppeteer виявився простішим для швидкого написання тестів, тому що всі дії описуються в одному JavaScript-файлі. Gauge виглядає складнішим на старті, бо потрібно окремо створювати специфікації та реалізовувати кожен крок. Проте Gauge має перевагу в читабельності: сценарії у

файлі .spec виглядають зрозуміло і можуть використовуватися як документація до тестів.

Після успішного запуску тестів я підготувала проєкт до завантаження на GitHub. У мене раніше уже був ініціалізований Git-репозиторій для виконання цих робіт.

Тому першим ділом я перевірила стан файлів:

```
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Невідстежувані файли:
  (скористайтесь "git add <файл>...", щоб додати до майбутнього коміту)
    lab-8-tools-comparison/

нічого не додано до коміту, але є невідстежувані файли (скористайтесь "git add" для відстежування)
oksanka@debian:~/Documents/automated_testing$
```

Далі я додала всі файли до індексації Git:

```
oksanka@debian:~/Documents/automated_testing$ git add .
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Зміни, додані до майбутнього коміту:
  (використовуйте "git restore --staged <файл>...", щоб видалити з індексу)
    новий файл:    lab-8-tools-comparison/.dockerignore
    новий файл:    lab-8-tools-comparison/.gitignore
    новий файл:    lab-8-tools-comparison/.vscode/extensions.json
    новий файл:    lab-8-tools-comparison/Dockerfile
    новий файл:    lab-8-tools-comparison/env/default/default.properties
    новий файл:    lab-8-tools-comparison/env/default/headless.properties
    новий файл:    lab-8-tools-comparison/env/default/js.properties
    новий файл:    lab-8-tools-comparison/manifest.json
    новий файл:    lab-8-tools-comparison/package-lock.json
    новий файл:    lab-8-tools-comparison/package.json
    новий файл:    lab-8-tools-comparison/puppeteer-tests/vivat.puppeteer.test.js
    новий файл:    lab-8-tools-comparison/specs/vivat.spec
    новий файл:    lab-8-tools-comparison/tests/step_implementation.js
oksanka@debian:~/Documents/automated_testing$
```

Після цього я створила коміт:

```
oksanka@debian:~/Documents/automated_testing$ git commit -m "Add Puppeteer and Gauge UI tests for lab 8"
[main 078d4c1] Add Puppeteer and Gauge UI tests for lab 8
13 files changed, 5341 insertions(+)
create mode 100644 lab-8-tools-comparison/.dockerignore
create mode 100644 lab-8-tools-comparison/.gitignore
create mode 100644 lab-8-tools-comparison/.vscode/extensions.json
create mode 100644 lab-8-tools-comparison/Dockerfile
create mode 100644 lab-8-tools-comparison/env/default/default.properties
create mode 100644 lab-8-tools-comparison/env/default/headless.properties
create mode 100644 lab-8-tools-comparison/env/default/js.properties
create mode 100644 lab-8-tools-comparison/manifest.json
create mode 100644 lab-8-tools-comparison/package-lock.json
create mode 100644 lab-8-tools-comparison/package.json
create mode 100644 lab-8-tools-comparison/puppeteer-tests/vivat.puppeteer.test.js
create mode 100644 lab-8-tools-comparison/specs/vivat.spec
create mode 100644 lab-8-tools-comparison/tests/step_implementation.js
oksanka@debian:~/Documents/automated_testing$
```

Потім я завантажила проєкт на GitHub:

```
oksanka@debian:~/Documents/automated_testing$ git push -u origin main
Username for 'https://github.com': OksanaMykytyn
Password for 'https://OksanaMykytyn@github.com':
Перерахування об'єктів: 23, готово.
Підрахунок об'єктів: 100% (23/23), готово.
Дельта компресія з використанням до 4 потоків
Компресія об'єктів: 100% (18/18), готово.
Запис об'єктів: 100% (22/22), 48.57 KiB | 48.57 MiB/c, готово.
Всього 22 (дельта 1), повторно використано 21 (дельта 0), повторно використано пакунків
0 (з 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/OksanaMykytyn/automated_testing.git
06d9a76..078d4c1  main -> main
Гілку "main" налаштовано на відстежування "origin/main".
oksanka@debian:~/Documents/automated_testing$
```

Посилання на GitHub:

https://github.com/OksanaMykytyn/automated_testing

Висновок: під час лабораторної роботи я дослідила два інструменти для автоматизації UI-тестування: Puppeteer та Gauge. Я порівняла їхні сильні й слабкі сторони, а також визначила типові задачі, для яких вони підходять. Для практичної частини я обрала сайт Видавництва Віват і створила по три UI-тести з використанням кожного інструменту. Під час роботи з Puppeteer я побачила, що цей інструмент зручний для прямого керування браузером і швидкого написання технічних перевірок. Під час роботи з Gauge я зрозуміла, що його основна перевага полягає в читабельному описі сценаріїв і відокремленні тестової логіки від реалізації кроків. Я переконалася, що Puppeteer краще підходить для швидкої технічної автоматизації, а Gauge — для сценаріїв, які мають бути зрозумілими для ширшої команди.